딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사

목차

A table of Contents

01 지난시간 복습

02 Cross Entropy

03 역전파와 optimizer

04 학습과 검증 루프

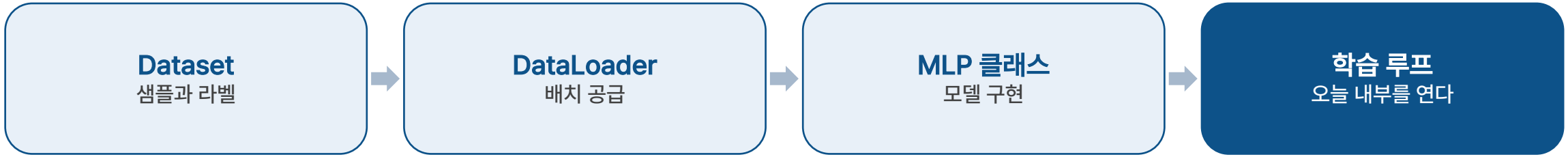
05 학습 곡선

06 Baseline

PART 01

지난시간 복습

1. 복습: 데이터와 모델은 직접 만들었다



- **지난주까지 완성한 것**
  - nn.Module 틀로 MLP를 직접 구현하고, 파이프라인에 연결해 학습
  - ReLU를 빼면 표현력이 한 계층 수준으로 줄어드는 것도 실험으로 확인
- **오늘의 질문**
  - 그런데 학습 루프 네 줄은 정확히 무슨 일을 할까?

1. 직선 맞추기와 같은 순서

	직선 맞추기	MNIST 숫자 분류
데이터	점 50개	사진 50,000장, 배치 64장씩
모델	직선 y = w·x + b	MLP 784-100-10
파라미터	w, b 2개	**79,510개**
loss	MSE, 차이 제곱의 평균	**Cross Entropy**
기울기 계산	`loss.backward()`	`loss.backward()`
갱신	`w -= 0.5 * w.grad` 직접 계산	`opt.step()`, SGD lr 0.1
기울기 지우기	`w.grad.zero_()`	`opt.zero_grad()`

- 파라미터가 2개에서 79,510개로 늘고 loss가 바뀌었을 뿐, **예측 → loss → backward → 한 걸음**의 순서는 같다

1. 오늘의 목표: 네 줄을 열고 기준점을 세운다

1주차부터 복사해 온 네 줄, 오늘 하나씩 연다

```
for x, y in train_loader:                          # 학습 루프
    x, y = x.view(-1, 784).to(device), y.to(device)
    loss = loss_fn(model(x), y)      # ① 얼마나 틀렸나
    opt.zero_grad()                  # ② 지난 기울기 비우기
    loss.backward()                  # ③ 기울기 계산
    opt.step()                       # ④ 파라미터 갱신
```

- **오늘 할 일 세 가지**
  - loss가 틀린 정도를 **숫자 하나로** 만드는 방법을 연다
  - backward와 step이 **79,510개 파라미터**를 고치는 과정을 확인한다
  - 학습과 검증을 기록해 앞으로의 모든 실험을 재는 **Baseline**을 세운다

1. 데이터

준비 셀 1. 학습용 50,000장과 검증용 10,000장, 로더 두 개

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
device = "cuda" if torch.cuda.is_available() else "cpu"
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())
g = torch.Generator().manual_seed(0)            # 2주차와 같은 분리
train_set, val_set = random_split(train_all, [50000, 10000], generator=g)
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
val_loader = DataLoader(val_set, batch_size=256, shuffle=False)
print(len(train_loader), len(val_loader))      # → 782 40
```

1. 준비 셀 2: 모델과 loss

준비 셀 2. 3주차 MLP에서 두 곳이 바뀐다

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flat = nn.Flatten()          # 바뀐 곳 1 (N, 1, 28, 28) → (N, 784)
        self.fc1 = nn.Linear(784, 100)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(100, 10)
    def forward(self, x):
        x = self.flat(x)                  # 바뀐 곳 2, 먼저 편다
        return self.fc2(self.act(self.fc1(x)))

loss_fn = nn.CrossEntropyLoss()
print(MLP()(torch.zeros(2, 1, 28, 28)).shape)   # → torch.Size([2, 10])
```

- **바뀐 두 곳**
  - `__init__`에 `nn.Flatten()`을 선언하고, `forward`에서 그것으로 **먼저 편다**
  - 이제 펴기는 모델이 하므로 학습과 검증 코드에서 `x.view(-1, 784)`가 빠지고, CNN에도 **같은 코드를 그대로** 쓴다
- **확인**
  - 사진 모양 (2, 1, 28, 28)을 넣어 `torch.Size([2, 10])`이 나오면 두 곳 모두 바뀐 것이다

PART 02

Cross Entropy로 틀린 정도를 잰다

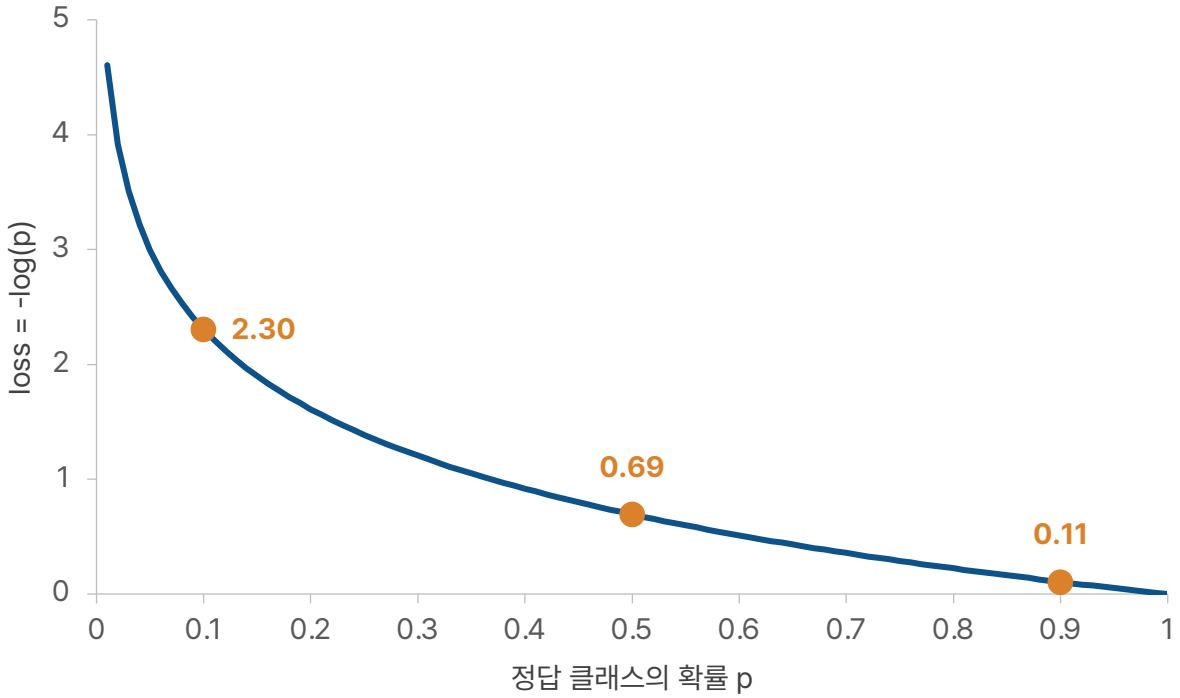
2. logits에서 loss까지



logits → softmax → 정답 클래스의 확률 → -log, 이 계산 전체를 한 번에 하는 것이 nn.CrossEntropyLoss다

- 모델의 출력 점수(**logits**)는 **softmax**를 거쳐 합이 1인 확률이 되며, 점수 순위는 그대로다
- **Cross Entropy**는 정답 클래스의 확률 p 하나만 보고 **-log(p)**를 loss로 매긴다 (log는 자연로그)
- 예시는 클래스 3개짜리 축소판이고, MNIST는 클래스가 10개일 뿐 계산은 같다

2. Cross Entropy



정답 확률이 0에 가까워질수록 loss가 가파르게 커진다

정답 확률 p	loss	정답 확률의 의미
0.99	0.01	정답에 거의 전부를 걸었다
0.9	0.11	정답 쪽으로 기울었다
0.5	0.69	정답에 절반을 걸었다
0.1	**2.30**	10개에 고르게 나눈 것과 같은 loss
0.01	4.61	정답을 거의 배제했다

- 정답 확률이 1에 가까우면 loss는 0에 가깝고, 정답 확률이 작을수록 loss가 가파르게 커진다 (log는 자연로그)

2. 예제 4-1: softmax와 Cross Entropy 직접 계산

예제 4-1. 손으로 한 계산과 nn.CrossEntropyLoss 비교

```
import torch
import torch.nn as nn
logits = torch.tensor([[2.0, 1.0, 0.1]])   # 샘플 1개, 클래스 3개의 점수
target = torch.tensor([0])                  # 정답은 0번 클래스
p = torch.softmax(logits, dim=1)            # 점수 → 확률
print(p)                        # → tensor([[0.6590, 0.2424, 0.0986]])
print(-torch.log(p[0, 0]))   # → tensor(0.4170)   정답 확률의 -log
loss_fn = nn.CrossEntropyLoss()
print(loss_fn(logits, target))   # → tensor(0.4170)   같은 값, softmax와 -log를 한 번에
```

- softmax 결과는 0.66, 0.24, 0.10으로 합이 1이고, 정답 확률의 -log와 `nn.CrossEntropyLoss`가 **똑같이 0.4170**이다
- loss 함수 안에서 softmax를 계산하므로 **모델 끝에는 softmax를 붙이지 않는다**

2. 예제 4-2

예제 4-2. 학습 전 모델의 확률과 첫 loss

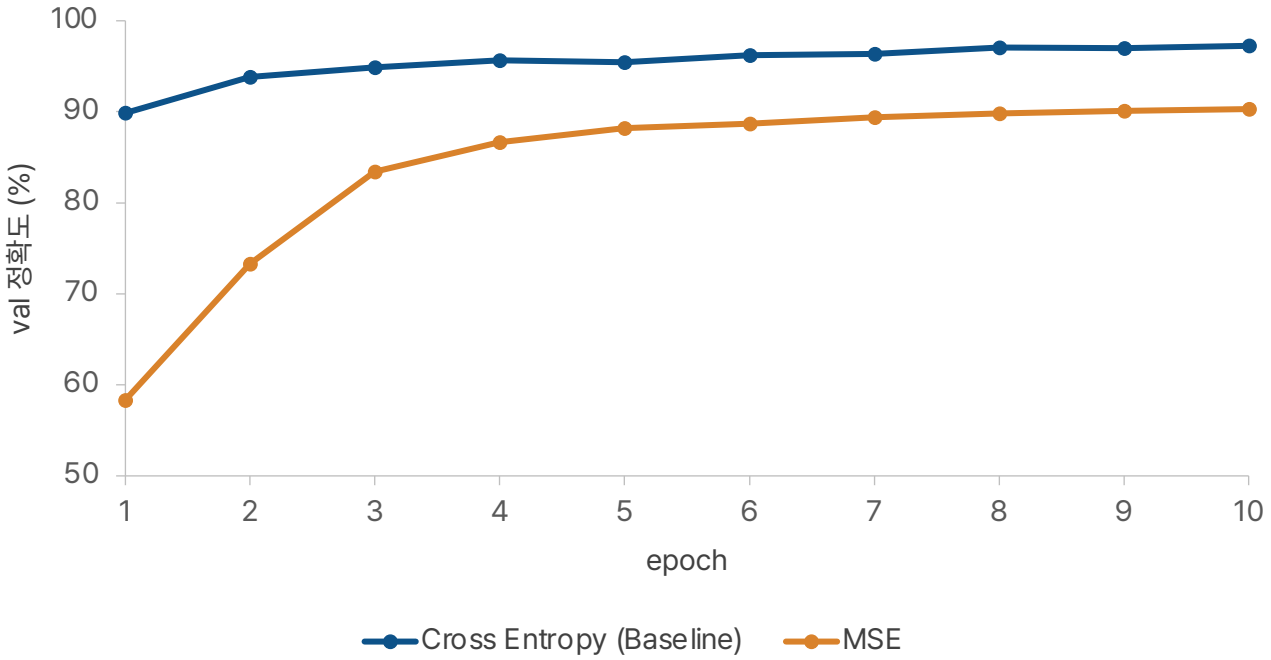
```
torch.manual_seed(0)
model = MLP().to(device)                          # 학습 전 모델
x, y = next(iter(train_loader))                   # 배치 하나 꺼내기
x, y = x.to(device), y.to(device)                 # 펴기는 이제 모델이 한다
p = torch.softmax(model(x), dim=1)                # 64개 샘플 × 10개 확률
print(round(p.min().item(), 2), round(p.max().item(), 2))
# → (예) 0.08 0.12
print(round(loss_fn(model(x), y).item(), 4))   # → (예) 2.2942   첫 loss
print(round(-torch.log(torch.tensor(0.1)).item(), 4))   # → 2.3026   (-log(1/10))
```

- 1주차 실습의 첫 loss 2.303은 학습 전 모델이 10개 클래스에 **거의 고르게 0.1씩** 확률을 주기 때문에 나온 값이다
- 배치의 loss는 64개 샘플 loss의 평균이라 **-log(0.1) = 2.303** 근처다, 이 값에서 크게 벗어나면 입력 전처리나 출력 계층을 먼저 의심한다

2. 분류에 MSE 대신 Cross Entropy를 쓰는 이유

정답 확률 p	Cross Entropy	MSE
0.9	0.11	0.001
0.5	0.69	0.028
0.1	2.30	0.090
0.01	4.61	0.109

- MSE는 확률 10개와 정답(정답 칸만 1)의 차이를 제곱해 평균한 값이다
- 크게 틀릴수록 Cross Entropy는 **끝없이 커지지만**, 정답 확률이 0.1 아래로 내려가도 MSE는 0.09~0.11에 머물러 **더 크게 틀린 것**을 거의 구분하지 못한다



같은 MLP와 설정에서 loss만 바꾼 실측 (SGD lr 0.1)

- loss만 MSE로 바꾸면 1 epoch 검증 정확도가 **58.4%**로 Cross Entropy(89.9%)보다 크게 낮다, 분류에는 Cross Entropy를 쓴다

2. 예제 4-3: 정확도는 같아도 loss는 다르다

예제 4-3. 맞힌 개수와 loss를 함께 보기

```
import torch
import torch.nn as nn
loss_fn = nn.CrossEntropyLoss()
labels = torch.tensor([0, 1, 2, 0])                          # 샘플 4개의 정답
out_a = torch.tensor([[4.0, 0.0, 0.0], [0.0, 4.0, 0.0],     # 모델 A, 확신하며 맞힌다
                      [0.0, 0.0, 4.0], [0.0, 2.0, 0.0]])
out_b = torch.tensor([[1.0, 0.8, 0.0], [0.8, 1.0, 0.0],     # 모델 B, 겨우 맞힌다
                      [0.0, 0.8, 1.0], [0.0, 2.0, 0.0]])
print((out_a.argmax(dim=1) == labels).sum().item(), (out_b.argmax(dim=1) == labels).sum().item())
# → 3 3   맞힌 개수는 같다
print(round(loss_fn(out_a, labels).item(), 4), round(loss_fn(out_b, labels).item(), 4))
# → 0.5869 1.1467   (loss는 B가 약 2배)
```

- 두 모델 모두 4개 중 3개를 맞혀 **정확도는 같지만**, 정답 확률이 낮은 B의 loss가 **약 2배**다
- 정확도는 맞았는지만 세고, loss는 **얼마나 확신하며** 맞혔는지까지 잰다, 그래서 4파트에서 둘 다 기록한다

2. 실습 1: loss 계산해 보기 (문제)

실습

1. 정답 확률이 0.9, 0.5, 0.1일 때의 loss를 `-torch.log(...)`로 계산
2. 예제 4-1의 점수에서 정답 점수 2.0을 4.0으로 바꾸면 loss가 커질지 작아질지 예측한 뒤 확인하기
3. 원래 점수 [2.0, 1.0, 0.1]에서 정답만 2번 클래스로 바꾸면 loss는?

- **힌트**
  - `torch.log(torch.tensor([0.9, 0.5, 0.1]))`처럼 세 값을 한 번에 넣는다
- **확인**
  - 1의 결과가 Cross Entropy 장의 표 값과 같으면 성공

2. 실습 1: 정답 예시

실습 1. 정답 예시

```
import torch
import torch.nn as nn
print(-torch.log(torch.tensor([0.9, 0.5, 0.1])))   # 1 정답 확률별 loss
# → tensor([0.1054, 0.6931, 2.3026])
loss_fn = nn.CrossEntropyLoss()
logits = torch.tensor([[4.0, 1.0, 0.1]])            # 2 정답 점수를 4.0으로
print(loss_fn(logits, torch.tensor([0])))
# → tensor(0.0677)   (0.4170보다 작아졌다)
logits = torch.tensor([[2.0, 1.0, 0.1]])            # 3 원래 점수에서
print(loss_fn(logits, torch.tensor([2])))           #    정답만 2번으로
# → tensor(2.3170)
```

- **포인트**
  - 정답 확률이 0.9, 0.5, 0.1로 내려갈수록 loss는 0.11, 0.69, 2.30으로 커진다
  - 정답 점수를 올리면 정답 확률이 커져 loss가 **0.4170에서 0.0677로** 줄어든다, 학습은 정답 확률이 커지는 쪽으로 점수들을 바꾼다
  - 가장 낮은 점수에 정답이 있으면 loss는 **2.3170**, 크게 틀린 만큼 크게 받는다

2. 예제 문제, loss 값 읽기

문제

1. 클래스가 100개인 분류 모델의 학습 전 첫 loss는 2.30보다 클까, 작을까? 이유는?
2. 정답 확률이 0.1 → 0.2로 오를 때와 0.5 → 0.6으로 오를 때, loss가 더 크게 줄어드는 쪽은?
3. forward 마지막에 softmax를 넣은 모델을 CrossEntropyLoss로 학습하면 무엇이 문제일까?

2. 예제 문제 풀이

실행해서 검증

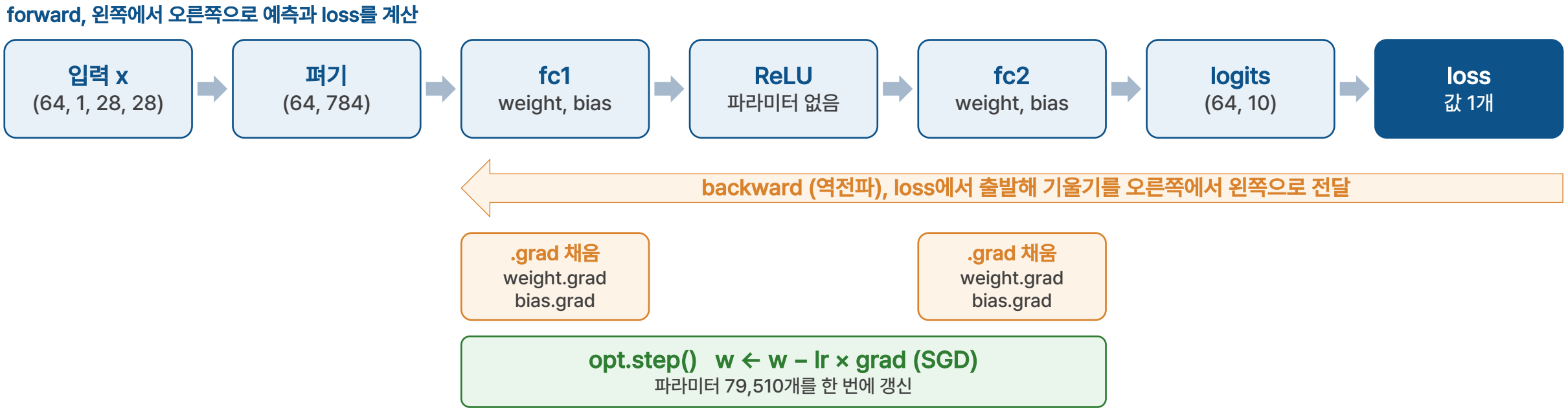
```
import torch
import torch.nn as nn
print(-torch.log(torch.tensor(1 / 100)))     # 1 클래스 100개
# → tensor(4.6052)
L = -torch.log(torch.tensor([0.1, 0.2, 0.5, 0.6]))
print(round((L[0] - L[1]).item(), 4))        # 2 0.1 → 0.2
# → 0.6931
print(round((L[2] - L[3]).item(), 4))        # 2 0.5 → 0.6
# → 0.1823
out = torch.zeros(1, 10)                     # 3 softmax를 거친 출력
out[0, 0] = 1.0                              #    정답 확률 1, 완벽한 경우
print(nn.CrossEntropyLoss()(out, torch.tensor([0])))
# → tensor(1.4612)   완벽해도 1.46 아래로 못 내려간다
```

- **풀이**
  - 1. 더 크다, 학습 전 출력이 100개에 거의 고르다면 0.01씩이라 -log(1/100) = **4.61** 근처다
  - 2. 0.1 → 0.2 쪽이 **0.69** 줄어 0.5 → 0.6의 0.18보다 크다
  - 3. softmax가 두 번 적용되어 loss가 **1.46 아래로 내려가지 못하고** 학습이 느려진다 (10 epoch 실측 92.87%, 정상은 97.25%)

PART 03

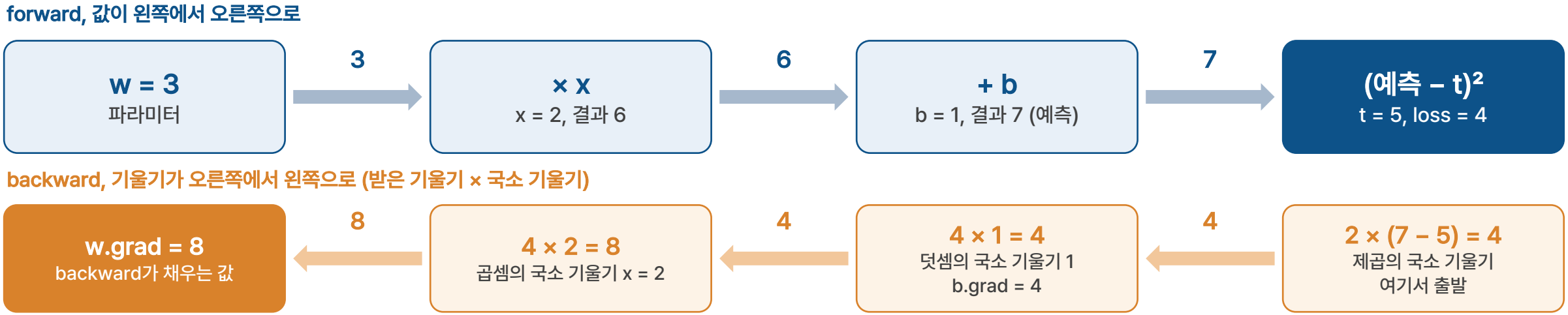
역전파와 optimizer로 파라미터를 고친다

3. 학습 한 step의 흐름: forward와 backward



- backward는 loss에서 출발해 **출력 쪽 계층부터 입력 쪽으로** 기울기를 전달하며 계산한다, 이것이 **역전파**다
- 각 계층은 받은 기울기로 자기 파라미터의 `.grad`를 채우고, 자기 입력 쪽 기울기를 앞 계층에 넘긴다

3. 연쇄 법칙: 기울기는 뒤에서부터 곱해 온다



- loss에서 출발해 각 단계가 **받은 기울기 × 국소 기울기**(그 단계만 따로 본 기울기)를 앞 단계로 넘긴다, 이 규칙이 **연쇄 법칙(chain rule)**이고 MLP의 79,510개도 backward 한 번에 이렇게 계산된다
- 국소 기울기는 외울 공식이 아니라 PyTorch가 연산마다 이미 아는 값이다 (제곱은 2 × (예측 – t), 덧셈은 1, 곱셈은 상대편 값)
- 기울기의 뜻으로 확인할 수도 있다, w를 3에서 3.01로 살짝 키우면 loss가 약 0.08(0.01 × 8) 커진다

3. 예제 4-4: 손으로 구한 기울기와 backward 비교

예제 4-4. 앞 장의 계산을 코드로

```
import torch
x, t = torch.tensor(2.0), torch.tensor(5.0)    # 입력 x와 정답 t
w = torch.tensor(3.0, requires_grad=True)      # 파라미터 두 개
b = torch.tensor(1.0, requires_grad=True)
pred = w * x + b                               # forward, 예측
loss = (pred - t) ** 2                         # forward, loss
loss.backward()                                # backward
print(pred.item(), loss.item())   # → 7.0 4.0
print(w.grad, b.grad)             # → tensor(8.) tensor(4.)   손으로 구한 8과 4
with torch.no_grad():             # 기울기 반대 방향으로 한 걸음
    w -= 0.05 * w.grad
    b -= 0.05 * b.grad
print(round(((w * x + b - t) ** 2).item(), 4))   # → 1.0   (loss가 4에서 1로)
```

- **손으로 구한 값과 같다**
  - backward가 채운 `w.grad` 8과 `b.grad` 4는 앞 장에서 손으로 구한 값과 같다
  - 기울기 반대 방향으로 한 걸음(lr 0.05) 옮기면 loss가 **4에서 1로** 줄었다
- **MLP에서는**
  - 같은 계산이 79,510개 파라미터에 대해 **한 번에** 일어난다 (예제 4-6)

3. 예제 4-5: logits의 기울기, 확률에서 정답을 뺀 값

예제 4-5. Cross Entropy의 backward가 logits에 남긴 값

```
import torch
import torch.nn as nn
loss_fn = nn.CrossEntropyLoss()
logits = torch.tensor([[2.0, 1.0, 0.1]], requires_grad=True)   # 예제 4-1의 점수
loss = loss_fn(logits, torch.tensor([0]))      # 정답은 0번
loss.backward()
print(torch.softmax(logits.detach(), dim=1))   # → tensor([[0.6590, 0.2424, 0.0986]])
print(logits.grad)                             # → tensor([[-0.3410,  0.2424,  0.0986]])
```

클래스	확률	정답	기울기
0 (정답)	0.659	1	**−0.341**
1	0.242	0	+0.242
2	0.099	0	+0.099

기울기 = 확률 − 정답

- logits의 기울기는 **확률에서 정답(정답 칸만 1)을 뺀 값**이다
- logits를 직접 갱신하면(실습 2) 정답 칸은 음수라 점수가 **오르고**, 나머지는 양수라 확률에 비례해 **내려간다**
- 정답 확률이 작을수록 정답 칸 기울기가 −1에 가까워져 **크게 틀린 샘플을 크게 고친다**

3. 예제 4-6: backward가 채우는 것

예제 4-6. 같은 seed로 예제 4-2의 모델과 배치를 다시 만들고 시작

```
torch.manual_seed(0)                          # 예제 4-2와 같은 모델, 같은 배치
model = MLP().to(device)
x, y = next(iter(train_loader))
x, y = x.to(device), y.to(device)
loss = loss_fn(model(x), y)
print(model.fc1.weight.grad)                  # → None   (아직 없음)
loss.backward()                               # 역전파 한 줄
print(model.fc1.weight.grad.shape)            # → torch.Size([100, 784])
print(model.fc2.bias.grad.shape)              # → torch.Size([10])
print(sum(p.grad.numel() for p in model.parameters()))   # → 79510
```

- backward 전에는 `grad`가 None이고, backward 한 줄 뒤에는 모든 파라미터에 **같은 shape의 grad**가 생긴다 (79,510개)
- 기울기는 파라미터를 조금 키웠을 때 loss가 변하는 정도다, 양수면 loss가 커진다는 뜻이니 **반대 방향**으로 고친다

3. 예제 4-7: step이 하는 일

예제 4-7. 예제 4-6에 이어서 실행

```
opt = torch.optim.SGD(model.parameters(), lr=0.1)   # 갱신 규칙 SGD
b0 = model.fc2.bias[0].item()            # 파라미터 하나의 갱신 전 값
g0 = model.fc2.bias.grad[0].item()       # 그 파라미터의 기울기
opt.step()                               # 79,510개를 한 번에 갱신
print(round(b0 - 0.1 * g0, 4))           # → (예) 0.0894   손으로 계산한 값
print(round(model.fc2.bias[0].item(), 4))    # → (예) 0.0894   (step 후 실제 값)
print(round(loss.item(), 4), round(loss_fn(model(x), y).item(), 4))
# → (예) 2.2942 2.2643   같은 배치의 loss가 줄었다
```

- SGD의 `step()`은 모든 파라미터에 **w ← w − lr × grad**를 한 번에 적용한다, 손으로 계산한 값과 step 후 실제 값이 같다
- 1주차 예제에서 직접 쓴 `w -= 0.5 * w.grad`와 같은 계산이고, 같은 배치의 loss가 조금 줄었다
- **optimizer**는 기울기를 받아 파라미터를 고치는 갱신 규칙이며, SGD가 기본이고 Adam과 비교한다

3. 예제 4-8: zero_grad가 필요한 이유

예제 4-8. 예제 4-7에 이어서 실행

```
opt.zero_grad()                          # 기울기 지우기
loss_fn(model(x), y).backward()          # 기울기 계산
g1 = model.fc2.bias.grad[0].item()
loss_fn(model(x), y).backward()          # zero_grad 없이 같은 계산을 한 번 더
g2 = model.fc2.bias.grad[0].item()
print(round(g2 / g1, 4))                 # → 2.0   덮어쓰지 않고 더해졌다 (누적)
```

- PyTorch는 backward 결과를 기존 grad에 **더한다(누적)**, 그래서 정확히 2배가 됐다
- 그래서 backward 전에 `opt.zero_grad()`로 지운다, 효과는 1주차 `w.grad.zero_()`와 같다 (지금은 0 대신 None으로 지운다)
- 지우지 않으면 지난 배치들의 기울기가 계속 섞여 현재 배치와 어긋난 방향으로 갱신된다, 결과는 예제 문제에서 확인한다

3. 실습 2: logits 3개를 네 줄로 학습시키기 (문제)

실습

1. 예제 4-5에서 정답을 2번 클래스로 바꾸면 세 기울기의 부호가 어떻게 될지 예측한 뒤 확인하기
2. `torch.optim.SGD([logits], lr=1.0)`로 opt를 만들어 한 걸음 갱신하고 loss가 줄었는지 확인하기
3. 네 줄을 10번 반복한 뒤 확률을 출력해 정답 확률 보기

- **왜 하나**
  - **logits(점수) 3개 자체를 파라미터**로 두면 네 줄이 하는 일이 숫자로 바로 보인다
- **힌트**
  - 정답은 `y`가 아닌 `target = torch.tensor([2])`로 만든다
  - opt는 `model.parameters()` 자리에 `[logits]`를 넣어 만든다
  - 1에서 backward를 했다면 2는 `opt.step()`만 부르고, `loss_fn(logits, target)`을 다시 계산해 비교한다
  - 확률은 `torch.softmax(logits.detach(), dim=1)`로 본다, `detach()`는 기울기 기록을 뗀 값만 준다

3. 실습 2: 정답 예시

실습 2. 정답 예시

```
logits = torch.tensor([[2.0, 1.0, 0.1]], requires_grad=True)   # logits 3개
target = torch.tensor([2])                                     # 1 정답을 2번으로
opt = torch.optim.SGD([logits], lr=1.0)                        # 파라미터 목록 자리에 [logits]
loss = loss_fn(logits, target)
opt.zero_grad(); loss.backward()
print(logits.grad)   # → tensor([[ 0.6590,  0.2424, -0.9014]])   (정답 칸만 음수)
opt.step()                                                     # 2 한 걸음
print(round(loss.item(), 4), round(loss_fn(logits, target).item(), 4))
# → 2.317 1.1594   (loss가 줄었다)
for step in range(10):                                         # 3 네 줄을 10번
    loss = loss_fn(logits, target)
    opt.zero_grad(); loss.backward(); opt.step()
print(torch.softmax(logits.detach(), dim=1))
# → tensor([[0.0361, 0.0305, 0.9334]])   (정답 확률이 0.93으로)
```

- **관찰**
  - 1. 정답 칸(2번)만 기울기가 음수다, 정답이 바뀌면 부호도 바뀐다
  - 2. 한 걸음에 loss가 **2.317에서 1.1594로** 줄었다, 정답 점수는 오르고 나머지는 내려갔다
  - 3. 10걸음 더(모두 11걸음) 가면 정답 확률은 **0.93**이다
- **모델 학습은**
  - 점수 대신 점수를 만드는 **79,510개 파라미터**를 같은 방식으로 고친다

3. 실습 3: 배치 하나 외우게 하기 (문제)

실습

1. 배치 하나를 고정하고 네 줄을 100번 반복하며 20 step마다 loss 출력하기
2. 그 배치 64개 중 몇 개를 맞히는지 확인하기
3. 검증 데이터에서 배치를 하나 꺼내 몇 개를 맞히는지 비교하기

- **왜 하나**
  - 정상이라면 배치 하나는 외울 수 있어야 한다, 못 외우면 버그를 의심하는 **실무 점검법**이다
- **힌트**
  - 예제 4-6처럼 새 모델을, 예제 4-7처럼 opt를 만들고, `next(iter(train_loader))`로 꺼낸 배치를 `for step in range(100):` 안에서 반복한다
  - 20 step마다 출력은 1주차처럼 `if step % 20 == 0:` 아래에 print를 넣는다
  - 맞힌 개수는 `(model(x).argmax(dim=1) == y).sum().item()`, ③의 배치는 `next(iter(val_loader))`로 꺼내 `.to(device)`로 옮긴다

3. 실습 3: 정답 예시

실습 3. 정답 예시

```
torch.manual_seed(0)
model = MLP().to(device)                        # 새 모델과 opt로 시작
opt = torch.optim.SGD(model.parameters(), lr=0.1)
x, y = next(iter(train_loader))                 # 학습할 배치 하나 고정
x, y = x.to(device), y.to(device)
for step in range(100):                         # ① 같은 배치로 100 step
    loss = loss_fn(model(x), y)
    opt.zero_grad(); loss.backward(); opt.step()
    if step % 20 == 0:
        print(step, round(loss.item(), 4))      # (예) 0 2.2942 ... 80 0.209 (5줄)
print((model(x).argmax(dim=1) == y).sum().item())   # → (예) 64   (②)
x, y = next(iter(val_loader))                   # ③ 검증 데이터의 배치 (처음 보는 256개)
x, y = x.to(device), y.to(device)
print((model(x).argmax(dim=1) == y).sum().item(), len(y))   # → (예) 165 256
```

- **관찰**
  - 같은 배치를 반복하면 loss가 2.29에서 80 step에 0.209까지 내려가고, 100 step 뒤에는 **64개 중 64개**를 맞힌다, 적어도 학습 루프는 작동한다는 신호다
  - 검증 배치는 **256개 중 165개(64%)**만 맞힌다, 학습에 쓴 데이터를 맞히는 것과 처음 보는 데이터를 맞히는 것은 다르다
- **다음**
  - 이 간격이 **과적합**의 씨앗이다

3. 예제 문제, 학습 코드 진단

문제

1. 학습 루프에서 `opt.step()`을 빠뜨리면 어떻게 될까? 오류가 날까?
2. `opt.zero_grad()`를 빠뜨리고 1 epoch을 학습하면 검증 정확도는?
3. 실수로 `with torch.no_grad():` 블록 안에서 loss를 만들고 `loss.backward()`를 부르면?

3. 예제 문제 풀이

상황	1 epoch 뒤 검증 정확도 (실측)	이유
정상, 네 줄 그대로	89.87%	기준
1. step 누락	**8.74%**, 학습 전과 같음	기울기만 계산하고 파라미터를 고치지 않는다, 오류 없이 조용히 실패
2. zero_grad 누락	**12.66%**	지난 배치들의 기울기가 계속 더해져 현재 배치와 어긋난 갱신이 이어진다
3. no_grad 안에서 backward	실행 오류	기울기 추적 기록이 없는 loss라 역전파할 수 없다

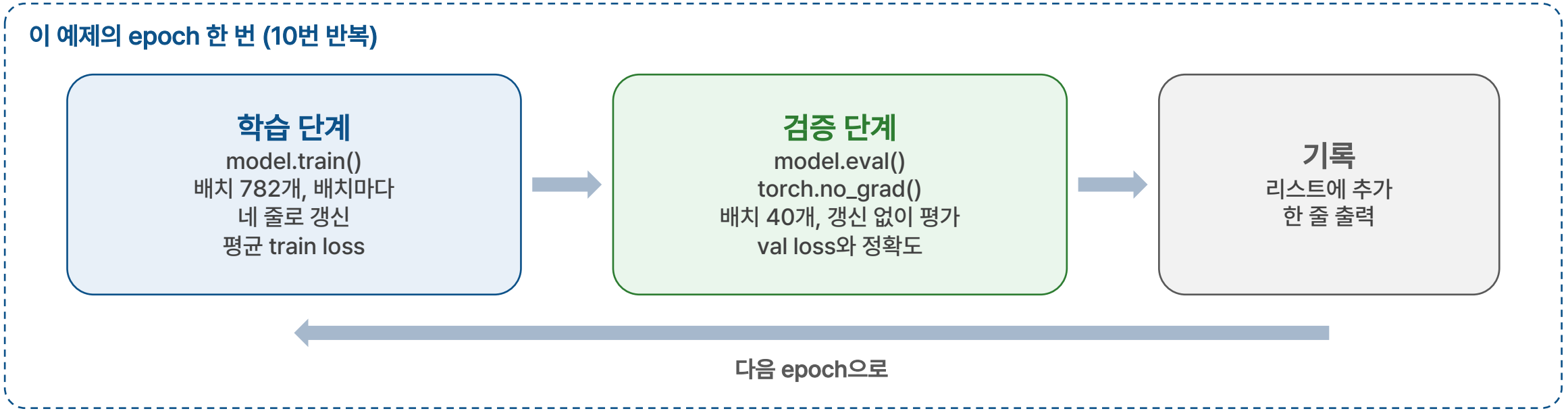
3을 직접 실행하면

```
import torch
w = torch.tensor(1.0, requires_grad=True)
with torch.no_grad():                    # 평가용 블록 안에서 loss를 만들면
    loss = (w * 3 - 1) ** 2
loss.backward()
# → RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn
```

PART 04

학습과 검증 루프를 표준 틀로 만든다

4. epoch마다 도는 세 단계



- `model.train()`과 `model.eval()`은 학습과 평가에서 다르게 동작하는 계층(Dropout, BatchNorm)을 **학습 모드와 평가 모드로 전환**한다
- 지금 MLP에는 그런 계층이 없어 결과가 같지만, **다음 시간부터 달라지므로** 지금부터 습관으로 넣는다

4. 예제 4-9: 검증 함수 evaluate

예제 4-9. 검증 함수 (표준 틀)

```python
def evaluate(model, loader):                # 검증 함수, 표준 틀이라 복사해서 쓴다
    model.eval()                            # 평가 모드로 전환
    total_loss, correct = 0.0, 0
    with torch.no_grad():                   # 기울기 계산 끄기
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            total_loss += loss_fn(out, y).item() * len(y)   # 평균 × 개수
            correct += (out.argmax(dim=1) == y).sum().item()
    n = len(loader.dataset)                 # 전체 샘플 수 (직접 쓰지 않는다)
    return total_loss / n, correct / n * 100
```

점검 셀, 학습 전 모델로 확인

```python
torch.manual_seed(0)
vl, va = evaluate(MLP().to(device), val_loader)   # 학습 전 모델로 점검
print(round(vl, 4), round(va, 2))   # → (예) 2.3024 8.74   학습 전 수준이 나와야 한다
```

- **읽는 법**
  - `def`로 시작하는 블록은 이름을 붙인 코드 묶음이다, 표준 틀이니 그대로 복사한다
  - 배치 loss는 배치 평균이라 **샘플 수를 곱해 더한 뒤** 전체로 나누고, 분모는 `len(loader.dataset)`을 쓴다
- **점검**
  - 학습 전 모델이 loss 2.30, 정확도 약 10%면 큰 실수는 없다는 신호다

4. 예제 4-10: 학습 함수 train_one_epoch

예제 4-10. 학습 함수 (표준 틀)

```python
def train_one_epoch(model, loader, opt):    # 학습 함수, 1 epoch을 돈다
    model.train()                           # 학습 모드로 전환
    total_loss = 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        loss = loss_fn(model(x), y)         # 1. 손실
        opt.zero_grad()                     # 2. 기울기 지우기
        loss.backward()                     # 3. 역전파
        opt.step()                          # 4. 갱신
        total_loss += loss.item() * len(y)
    return total_loss / len(loader.dataset)   # epoch 평균 loss
```

- **읽는 법**
  - 안쪽은 오늘 연 **네 줄 그대로**다, 앞에 학습 모드 전환, 뒤에 loss 기록만 붙었다
  - 반환값은 이번 epoch 동안 잰 **평균 train loss**다
- **참고**
  - train loss는 파라미터가 바뀌는 도중에 잰 평균이라, 학습 초반의 큰 loss가 섞여 보통 epoch이 끝난 시점보다 높게 나온다

4. 예제 4-11: epoch 루프와 기록

예제 4-11. 10 epoch 학습과 기록

```
torch.manual_seed(0)
model = MLP().to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1)
train_losses, val_losses, val_accs = [], [], []   # 기록용 빈 리스트 세 개
for epoch in range(10):
    tl = train_one_epoch(model, train_loader, opt)
    vl, va = evaluate(model, val_loader)
    train_losses.append(tl)                       # epoch마다 기록
    val_losses.append(vl)
    val_accs.append(va)
    print(epoch + 1, round(tl, 4), round(vl, 4), round(va, 2))
# → (예) 10 0.0702 0.0987 97.25   (10줄 중 마지막 줄, 전체는 오른쪽 표)
```

epoch	train loss	val loss	val 정확도
1	0.4794	0.3561	89.87%
2	0.2439	0.2164	93.82%
3	0.1862	0.1792	94.91%
4	0.1505	0.1546	95.68%
5	0.1271	0.1625	95.46%
6	0.1093	0.1289	96.23%
7	0.0964	0.1214	96.35%
8	0.0859	0.1079	97.03%
9	0.0775	0.1068	96.98%
10	0.0702	0.0987	97.25%

- 출력 한 줄이 epoch 하나의 기록이다, 10 epoch에 **97.25%**에 도달했다
- 기록이 리스트에 남아 있으니 5파트에서 바로 그래프로 그린다

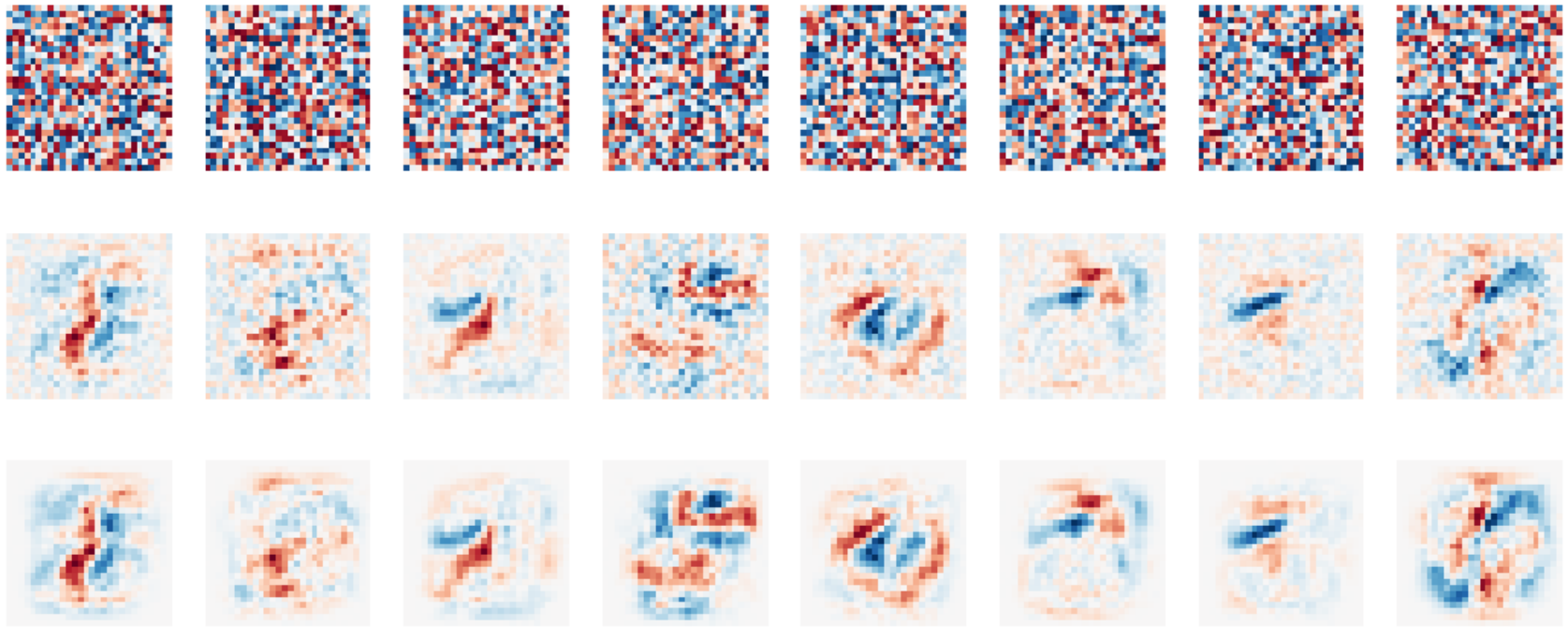
4. 예제 4-12: 학습 전후 weight 그려 보기

예제 4-12. 예제 4-11에서 학습한 model과 비교

```
import matplotlib.pyplot as plt
torch.manual_seed(0)
before = MLP()                          # 같은 seed라 학습 전 초기값과 같다
fig, axes = plt.subplots(3, 8, figsize=(12, 5))   # 학습 전, 학습 후, 변화량
for i in range(8):                      # 은닉 뉴런 0~7번
    w0 = before.fc1.weight[i].detach().view(28, 28)        # 784개 → 28×28
    w1 = model.fc1.weight[i].detach().cpu().view(28, 28)
    for r, w in enumerate([w0, w1, w1 - w0]):
        v = w.abs().max()               # 0이 흰색이 되도록 색 범위를 ±v로
        axes[r, i].imshow(w, cmap="RdBu_r", vmin=-v, vmax=v)
        axes[r, i].axis("off")
plt.show()
```

- **아이디어**
  - fc1의 뉴런 하나는 weight를 **784개** 가진다, 28×28로 펴면 그 뉴런이 **입력 사진의 어디를 보는지** 그림이 된다
  - 같은 seed로 새로 만든 모델은 학습 전 초기값과 같아서, 학습 전, 학습 후, 변화량을 나란히 볼 수 있다

4. 결과: weight에 획 모양이 생겼다



위부터 학습 전, 학습 후, 변화량 (은닉 뉴런 0~7번, 그림마다 색 범위 다름, 0은 흰색)

- 학습 전 잡음이던 weight에 **숫자의 획을 닮은 패턴**이 생겼다, 빨강은 양수, 파랑은 음수, 흰색은 0이다
- 변화량의 테두리가 하얗다, 늘 0인 가장자리 픽셀 68개는 기울기도 0이라 그 weight **6,800개는 한 번도 바뀌지 않았다**

4. 예제 문제, 학습 루프와 검증 루프 진단

문제

1. 학습 함수에서 `total_loss += loss.item() * len(y)`의 `.item()`을 빠뜨리면 어떻게 될까? 오류가 난다면 어디서?
2. 검증 함수의 분모를 `len(loader.dataset)` 대신 `len(loader)`로 쓰면 학습 전 점검 셀은 무엇을 출력할까?
3. 검증 함수에서 `model.eval()`과 `with torch.no_grad():`를 지워도 지금 MLP의 검증 결과는 같을까?

4. 예제 문제 풀이

상황	실측 결과	이유
1. `.item()` 누락	1 epoch은 끝까지 돌고 **예제 4-11의 출력 줄**에서 TypeError	반환값이 숫자가 아니라 tensor라서 round가 처리하지 못한다, 원인과 오류 위치가 다르다
2. 분모를 `len(loader)`로	loss 575.6, 정확도 **2,185%**	샘플 수 10,000 대신 배치 수 40으로 나눴다, 점검 셀이 바로 잡아낸다
3. eval과 no_grad 삭제	val loss와 정확도가 **똑같다** (89.87%)	eval()은 Dropout이 없는 지금 MLP에선 차이가 없고(6주차부터 달라진다), no_grad는 결과가 아니라 메모리에 영향을 준다

1을 실행한 결과 (.item()을 뺀 학습 함수로 1 epoch 학습한 뒤, GPU)

```
print(tl)            # .item()을 뺀 학습 함수가 돌려준 값
# → tensor(0.4794, device='cuda:0', grad_fn=<DivBackward0>)
print(round(tl, 4))  # 예제 4-11의 출력 줄에서
# → TypeError: type Tensor doesn't define __round__ method
```

PART 05

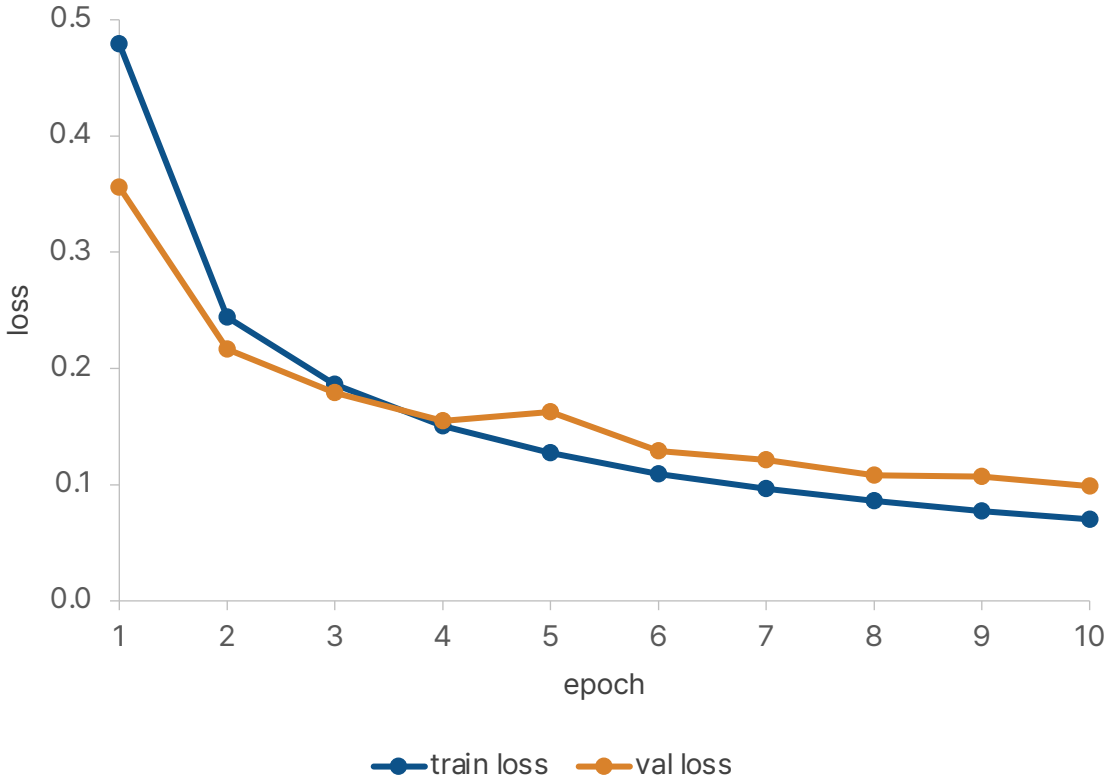
학습 곡선으로 모델 상태를 진단한다

5. 예제 4-13: 학습 곡선 그리기

예제 4-13. 예제 4-11의 기록으로 그리기

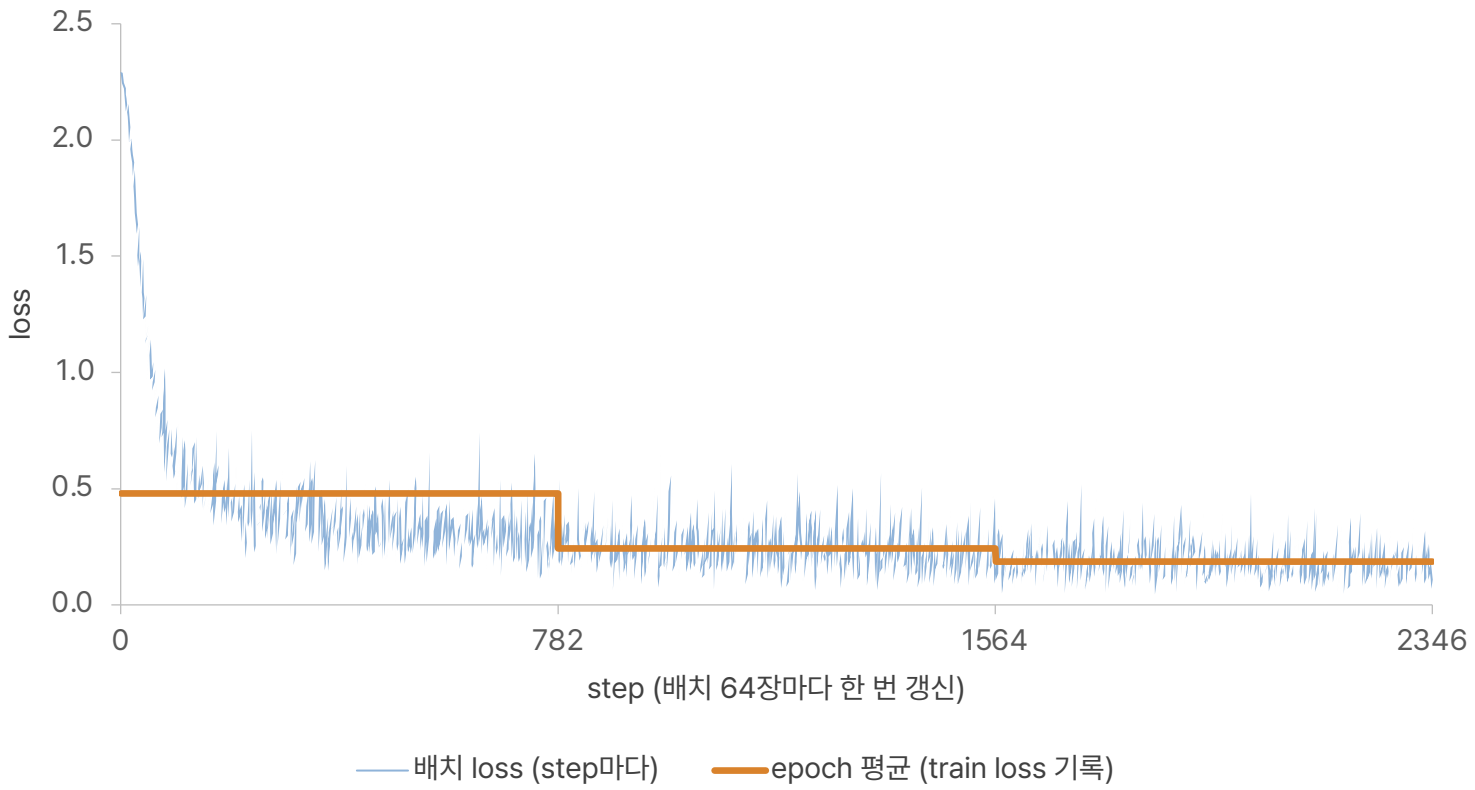
```
import matplotlib.pyplot as plt
ep = range(1, len(train_losses) + 1)      # epoch 번호
plt.plot(ep, train_losses, marker="o", label="train loss")
plt.plot(ep, val_losses, marker="o", label="val loss")
plt.xlabel("epoch"); plt.ylabel("loss")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

- 이 그래프를 **학습 곡선**이라 부른다, 학습 상태를 진단하는 가장 기본 도구다
- 두 곡선이 **함께 내려가고 간격이 작으면** 정상적으로 학습되는 중이다
- 첫 epoch의 train loss가 더 큰 것은 갱신 도중에 잰 평균이기 때문이다



예제 4-11의 기록 (실측)

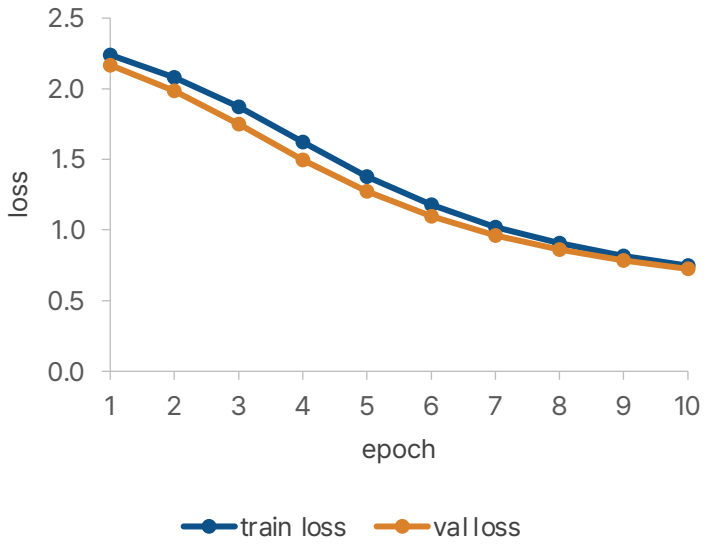
5. step마다 재면 loss는 흔들린다

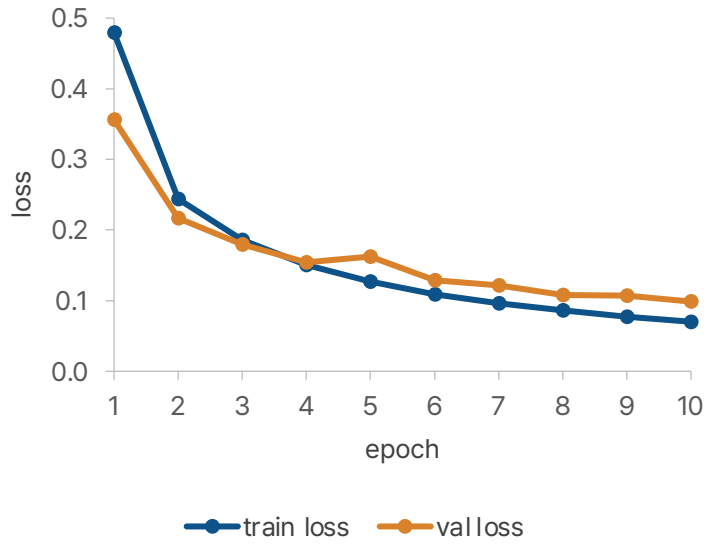


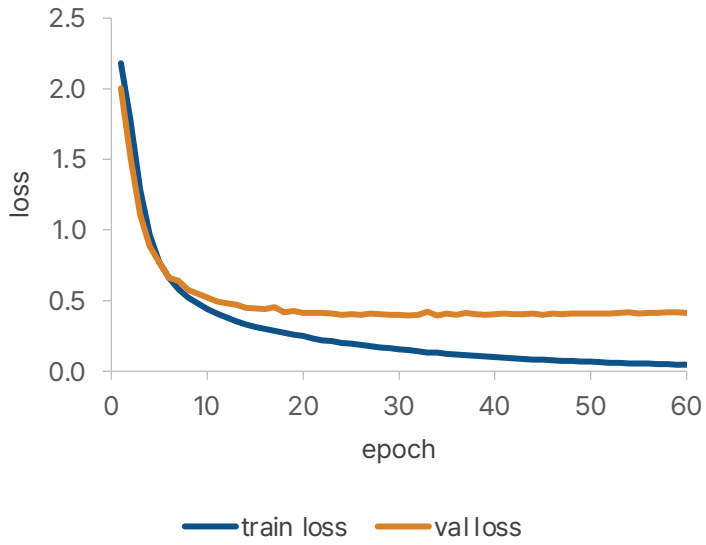
같은 학습의 처음 3 epoch (2,346 step) 실측

- 모델이 그대로여도 64장마다 쉬운 사진과 어려운 사진이 섞인 정도가 달라 **배치 loss는 흔들린다**
- SGD는 그 64장으로 구한 기울기로 한 걸음씩 간다, S(Stochastic, 확률적)가 이 뜻이고 흔들려도 평균적으로는 내려간다
- 그래서 학습 곡선에는 **epoch 평균**과 검증 데이터 전체의 loss를 기록한다

5. 곡선의 세 가지 모양







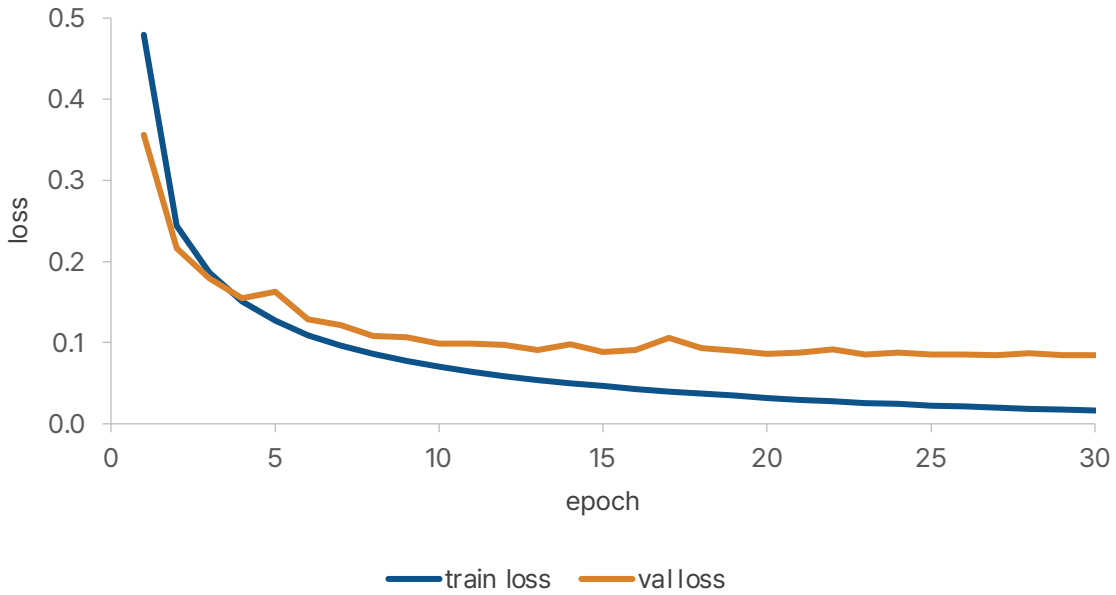
(A) 학습 부족, lr 0.001, 10 epoch
두 곡선이 아직 높고 천천히 내려간다
epoch을 늘리거나 lr을 점검한다

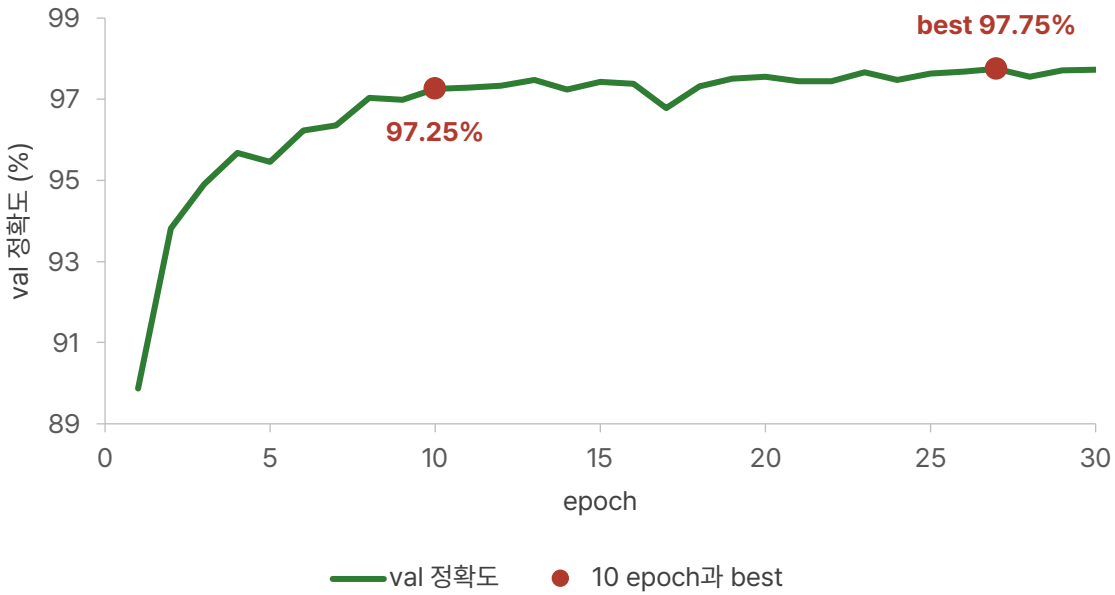
(B) 정상, Baseline, lr 0.1, 10 epoch
둘 다 내려가며 간격이 작다
그대로 진행, 정체되면 멈춘다

(C) 과적합, 학습 데이터 1,000장, 60 epoch
train만 계속 내려가고 val은 멈춘다
best 모델을 쓰고 6주차 대책을 쓴다

같은 MLP에서 조건만 바꾼 실측 학습 곡선, 세로축 범위가 그래프마다 다르다

5. 적정 epoch은 검증 곡선이 정한다





Baseline 조건 그대로 30 epoch까지 늘린 실측

- train loss는 끝까지 내려가지만 val 정확도는 20 epoch 이후 **97.4~97.8% 사이에서 오르내린다** (best 27 epoch)
- 적정 epoch은 **검증 곡선이 정체되는 지점**이고, 학습 중 가장 좋았던 순간의 모델을 저장해 둔다

5. 실습 4: 학습 데이터를 줄이면 (문제)

실습

1. 학습 데이터 앞 1,000장만 쓰는 `small_loader`로 30 epoch 학습하기 (예제 4-11에서 두 곳 변경)
2. 예제 4-13 코드를 빈 셀에 복사해 곡선을 그리고, 학습 데이터 정확도와 검증 정확도의 간격 재기
3. 5,000장으로 늘려 다시 학습하고 간격 비교하기

1 로더 만들기, 이 세 줄을 먼저 실행

```
from torch.utils.data import Subset
small_set = Subset(train_set, range(1000))          # 학습 데이터 앞 1,000장만
small_loader = DataLoader(small_set, batch_size=64, shuffle=True)
```

- 예제 4-11을 복사해 `train_loader`를 `small_loader`로, `range(10)`을 `range(30)`으로 바꾼다
- 기록 리스트 이름이 같으니 예제 4-13 코드를 **빈 셀에 복사**하면 그대로 그려진다, 원래 셀을 다시 실행하면 Baseline 그림이 지워진다
- 학습 데이터 정확도는 `evaluate(model, small_loader)`로 잰다, 검증 함수에 학습 로더를 넣을 뿐이다

5. 실습 4: 정답 예시

실습 4. 정답 예시, ①

```
from torch.utils.data import Subset
small_set = Subset(train_set, range(1000))          # 학습 데이터 앞 1,000장만
small_loader = DataLoader(small_set, batch_size=64, shuffle=True)
torch.manual_seed(0)                                # 여기부터 예제 4-11과 같다
model = MLP().to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1)
train_losses, val_losses, val_accs = [], [], []
for epoch in range(30):                             # 바꾼 곳 1, 30 epoch
    tl = train_one_epoch(model, small_loader, opt)  # 바꾼 곳 2, small_loader
    vl, va = evaluate(model, val_loader)
    train_losses.append(tl)
    val_losses.append(vl)
    val_accs.append(va)
    print(epoch + 1, round(tl, 4), round(vl, 4), round(va, 2))
# → (예) 30 0.1545 0.3996 88.33   (30줄 중 마지막 줄)
```

- **바꾼 곳**
  - 로더를 `small_loader`로, epoch을 30으로 바꾼 두 곳뿐이다
- **2의 곡선**
  - train loss는 30 epoch 내내 내려가는데 val loss는 0.40 근처에서 멈춘다
  - 학습 데이터 정확도와 간격은 다음 장에서 데이터 크기별로 비교한다

5. 실습 4: 결과, 데이터가 적을수록 간격이 크다

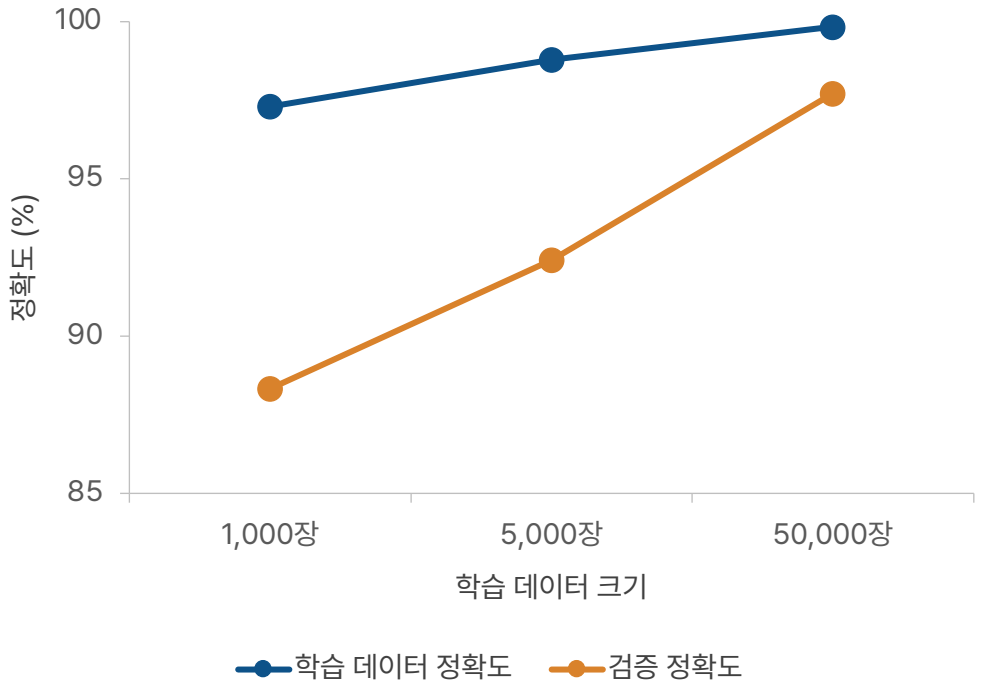
실습 4. 정답 예시, 2 학습 데이터 정확도

```
train_loss, train_acc = evaluate(model, small_loader)   # 2 학습 데이터 1,000장으로 평가
print(round(train_acc, 2), round(va, 2))   # → (예) 97.3 88.33   (학습 데이터와 마지막 epoch의 검증)
```

학습 데이터	학습 데이터 정확도	검증 정확도	간격
1,000장	97.30%	88.33%	**8.97%p**
5,000장	98.80%	92.43%	**6.37%p**
50,000장 (전체)	99.84%	97.72%	**2.12%p**

같은 MLP, 30 epoch의 실측, 데이터가 많을수록 갱신 횟수도 많다

- 데이터가 적으면 학습 데이터는 거의 다 맞히는데 검증에서 크게 뒤처진다, 이 **간격**이 과적합을 알려 주는 가장 직접적인 단서다
- 데이터를 늘리면 간격이 줄고 검증 정확도도 오른다, 데이터를 다루는 방법은 **다음 시간에서 다룬다**



두 선 사이가 학습 데이터와 검증의 간격 (실측)

5. 예제 문제, 학습 로그 진단

문제 아래 세 학습 로그는 각각 어떤 상태이고, 무엇을 바꿔야 할까? (모두 같은 MLP의 실측 로그)

로그	epoch	train loss	val loss	val 정확도
참고, Baseline	8 → 9 → 10	0.086 → 0.077 → 0.070	0.108 → 0.107 → 0.099	97.0% → 97.0% → 97.2%
1	8 → 9 → 10	0.297 → 0.286 → 0.276	0.298 → 0.289 → 0.280	91.7% → 91.9% → 92.1%
2	17 → 30 → 40	0.096 → 0.028 → 0.014	0.345 → 0.371 → 0.398	89.8% → 90.2% → 90.0%
3	1 → 2 → 3	2.59 → 2.33 → 2.33	2.57 → 2.36 → 2.46	10.9% → 9.8% → 10.8%

- 곡선의 세 가지 모양에 하나가 더 있습니다, 1주차 학습률 실험과 2파트의 2.303을 떠올려 보세요

5. 예제 문제 풀이

로그	진단	조치	실제 조건
1	**학습 부족**, Baseline보다 loss가 크고 아직 천천히 내려가는 중	epoch을 늘리거나 lr을 키워 본다	SGD lr 0.01
2	**과적합**, train loss는 내려가는데 val loss가 다시 오른다	검증 정확도가 가장 높던 epoch의 모델을 쓰고 6주차 대책	학습 데이터 2,000장, Adam
3	**학습 붕괴**, loss가 2.30 위에서 멈추고 정확도가 무작위 수준	lr을 낮춰 **처음부터 다시** 학습한다	SGD lr 5.0

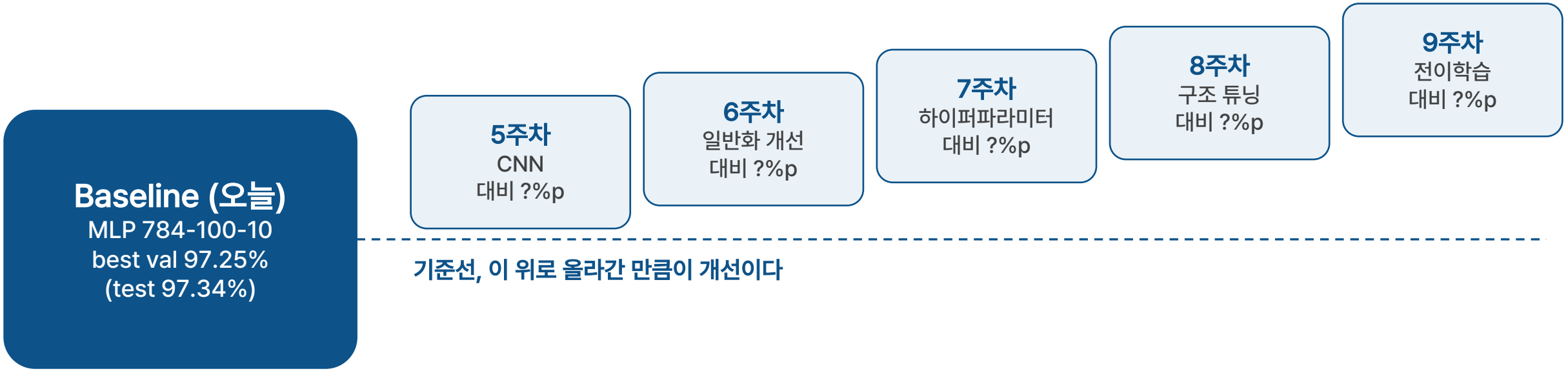
곡선과 로그를 보면 무엇을 바꿔야 할지 보인다, 하이퍼파라미터 튜닝의 출발점

- 3.을 실측하면 검증 이미지 10,000장에서 은닉 뉴런 100개가 **모두 꺼져** 전부 같은 클래스를 답했다, lr을 0.1로 낮춰 이어 학습해도 11.0%라 처음부터 다시 한다
- 2.에서 loss만 오른 이유는 틀린 샘플의 평균 loss가 2.7에서 3.6으로 커졌기 때문이다, 틀린 답에 점점 더 확신하게 됐다
- lr이 너무 크면 1주차처럼 nan으로 튕겨 나가거나, 3.처럼 한 클래스만 고르는 상태로 굳어 버린다

PART 06

기준점이 되는 Baseline을 세운다

6. Baseline: 모든 개선을 재는 기준점



- **Baseline**은 이후 모든 개선을 비교하는 기준점이며, 조건을 고정해 두어야 **무엇 때문에 좋아졌는지** 말할 수 있다
- 기준선은 **검증 정확도**이고, 차이는 **%p(퍼센트포인트)**로 쓴다, 정확도가 97.0%에서 97.5%가 되면 +0.5%p다

6. 예제 4-14: Baseline 학습과 best 모델 저장

예제 4-14. Baseline 학습, 검증 최고일 때만 저장

```
torch.manual_seed(0)                          # 조건 고정, seed
model = MLP().to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1)
best_acc, best_epoch = 0.0, 0
for epoch in range(10):
    train_one_epoch(model, train_loader, opt)
    vl, va = evaluate(model, val_loader)
    if va > best_acc:                         # 검증 최고 기록이면 저장
        best_acc, best_epoch = va, epoch + 1
        torch.save(model.state_dict(), "baseline_mlp.pt")
print(best_epoch, round(best_acc, 2))         # → (예) 10 97.25
```

- **Baseline 규칙 세 가지**
  - seed 0, 같은 데이터 분리, SGD lr 0.1, batch 64, 10 epoch(수업 시간에 맞춘 값)으로 **조건을 고정**한다
  - 모델 선택은 **검증 정확도**로 하고, 최고일 때만 `state_dict`(파라미터 묶음)를 저장한다
  - 성능 보고는 **테스트 세트로 한 번** 한다 (예제 4-16)
- **왜 best를 저장하나**
  - 마지막 epoch이 최고라는 보장이 없다, seed 3 에서는 9 epoch 97.17%가 10 epoch에 96.92%로 떨어졌다

6. 예제 4-15: 저장한 파일에는 무엇이 들어 있나

예제 4-15. 저장한 best 파일 열어 보기

```
import os
state = torch.load("baseline_mlp.pt", map_location=device)   # 저장한 파일 열기
for name, w in state.items():                    # 계층 이름과 파라미터 묶음
    print(name, tuple(w.shape))
# → fc1.weight (100, 784)
# → fc1.bias (100,)
# → fc2.weight (10, 100)
# → fc2.bias (10,)
print(sum(w.numel() for w in state.values()))  # → 79510   (예제 4-6에서 센 개수와 같다)
print(os.path.getsize("baseline_mlp.pt"))      # → (예) 320527   (바이트, 약 0.3MB)
```

- 파일에는 계층 이름별 weight와 bias, 즉 **파라미터 숫자 79,510개만** 들어 있다 (약 0.3MB)
- 파라미터가 없는 `flat`, `act`와 모델 구조(클래스 코드)는 저장되지 않는다
- 그래서 불러올 때는 **같은 MLP 클래스로 모델을 먼저 만들고** 그 위에 덮어쓴다 (다음 장)

6. 예제 4-16: 테스트 세트로 최종 평가

예제 4-16. 저장한 best 모델을 불러와 테스트

```
test_set = datasets.MNIST(root="data", train=False, download=True,
                          transform=transforms.ToTensor())   # 처음 보는 10,000장
test_loader = DataLoader(test_set, batch_size=256, shuffle=False)
model = MLP().to(device)                                     # 새 모델(초기값)에
state = torch.load("baseline_mlp.pt", map_location=device)  # 저장한 best 파라미터를
model.load_state_dict(state)                                 # 덮어쓴다
test_loss, test_acc = evaluate(model, test_loader)
print(len(test_set), round(test_acc, 2))     # → (예) 10000 97.34
```

- 테스트 세트는 학습에도, epoch 선택에도 쓰지 않은 데이터라 **처음 보는 실력**을 보여 준다
- 테스트는 **마지막에 한 번만** 본다, 테스트 점수를 보며 설정을 고치면 테스트가 검증으로 변해 버린다
- 새 모델의 초기값을 저장한 파라미터로 덮어쓰는 이 방식은 9주차 사전학습 모델에서도 그대로 쓴다

6. Baseline 기록표

항목	Baseline 기록
데이터	MNIST, train 50,000 / val 10,000 / test 10,000
모델	MLP 784-100-10, 파라미터 79,510개
학습 설정	SGD lr 0.1, batch 64, 10 epoch, seed 0
모델 선택	검증 정확도 최고 epoch (10 epoch)
결과	best val **97.25%**, test **97.34%**
학습 시간	epoch당 약 5초 (RTX A5000, 10 epoch 약 1분)

오늘의 Baseline 기록

- 비교와 채택 판단은 **best val**로 하고, test는 실험마다 마지막에 한 번 재서 **기록만** 한다

6. 실습 5: 첫 비교 실험, optimizer 교체 (문제)

실습

1. 예제 4-14, 4-16의 결과로 Baseline 줄 채우기
2. 예제 4-14를 복사해 opt를 `torch.optim.Adam(model.parameters(), lr=0.001)`로, 저장 파일을 바꿔 학습하고 테스트하기
3. 검증 기준으로 Baseline 대비 몇 %p인지 계산

실험	바꾼 것	best val (epoch)	test	Baseline 대비 (val)
Baseline	없음			기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001			

- 테스트는 예제 4-16 참고
- **파일이름은 반드시 변경**, Baseline 파일을 덮어씌워짐
- Adam은 파라미터마다 갱신 폭을 조절하는 optimizer다, lr은 여전히 정해 줘야 하며 자세한 비교는 이후에

6. 실습 5: 정답 예시

실습 5. 정답 예시, 학습

```
torch.manual_seed(0)                                    # Baseline과 같은 seed
model = MLP().to(device)
opt = torch.optim.Adam(model.parameters(), lr=0.001)    # 바꾼 곳 1, optimizer 설정
best_acc, best_epoch = 0.0, 0
for epoch in range(10):
    train_one_epoch(model, train_loader, opt)
    vl, va = evaluate(model, val_loader)
    if va > best_acc:
        best_acc, best_epoch = va, epoch + 1
        torch.save(model.state_dict(), "exp1_adam.pt")  # 바꾼 곳 2, 파일 이름
print(best_epoch, round(best_acc, 2))                   # → (예) 10 97.43
```

이어서 테스트, 예제 4-16에서 파일 이름만 교체

```
state = torch.load("exp1_adam.pt", map_location=device)   # 예제 4-16에서 파일 이름만
model.load_state_dict(state)
test_loss, test_acc = evaluate(model, test_loader)
print(len(test_set), round(test_acc, 2))                  # → (예) 10000 97.59
```

- **바꾼 곳**
  - optimizer 설정 한 줄과 저장 파일 이름 두 곳이고, 테스트도 같은 파일 이름으로 불러온다
- **주의**
  - SGD는 lr 0.1, Adam은 흔히 쓰는 기본값 lr 0.001로 비교했다, 그래서 결론은 **이 설정 묶음**에 대한 것이다

6. 실습 5: 결과 비교

실험	바꾼 것	best val (epoch)	test	Baseline 대비 (val)
Baseline	없음	97.25% (10)	97.34%	기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001	97.43% (10)	97.59%	**+0.18%p**

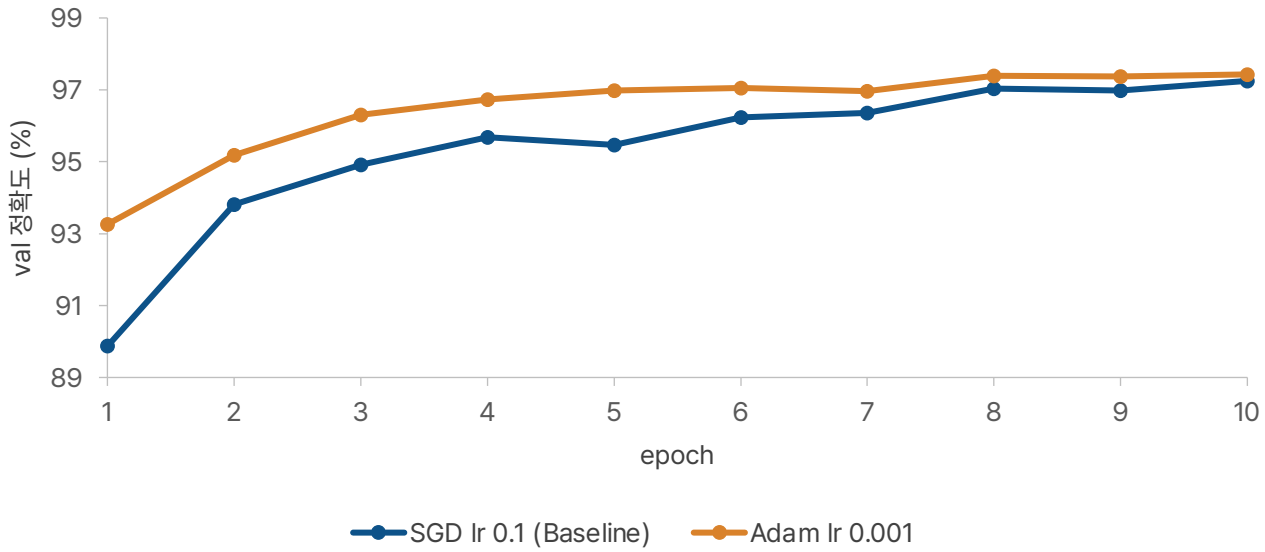


Fig. 9. epoch별 검증 정확도 (실측)

- Adam은 1 epoch에 93.3%로 **초반이 빠르고**, 10 epoch에서는 차이가 작다
- 판단은 검증 기준 **+0.18%p**로 하고, 테스트 +0.25%p는 기록용이다
- 3의 결론은 '한 번 실행에서는 Adam lr 0.001이 검증 기준 +0.18%p 높고, 의미 있는 차이인지는 반복해서 확인한다'처럼 쓴다

6. 예제 문제, 공정한 비교 진단

문제

1. "Adam으로 바꾸고 epoch도 30으로 늘렸더니 Baseline보다 0.4%p 올랐다"고 보고했다, 무엇 덕분이라고 말할 수 있을까?
2. 테스트 정확도가 가장 높은 epoch의 모델을 골라 그 점수를 보고했다면 무엇이 문제일까?
3. seed(초기 weight와 배치 순서를 정하는 값)만 바꿔 Baseline을 다섯 번 돌렸더니 best 검증 정확도가 97.06~97.25%였다, 실습 5의 검증 차이 0.18%p는 의미가 있을까?

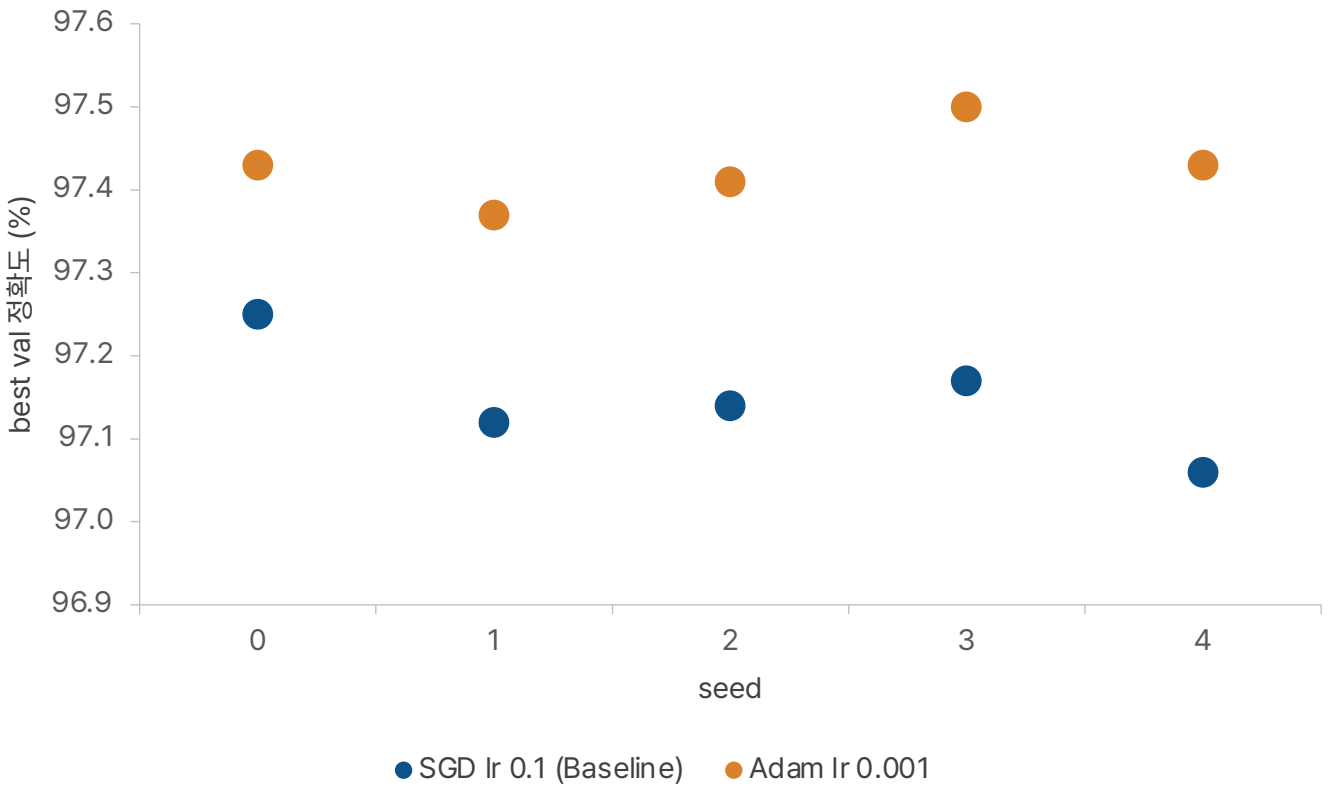
6. 예제 문제 풀이

문제	풀이	답
1	optimizer와 epoch을 동시에 바꿨다, SGD만 30 epoch으로 늘려도 best val이 +0.50%p 올랐다 (5파트 실측)	**한 번에 하나만** 바꾼다
2	테스트로 고르면 그 점수는 낙관적으로 부풀려진다	선택은 검증, **보고는 테스트로 한 번**
3	한 번의 차이 0.18%p는 seed 변동 폭 0.19%p와 비슷하다	한 번으로는 판단 못 한다, **반복해서 비교**한다

두 가지를 동시에 바꾸거나, 테스트로 고르거나, 한 번만 재면 결론을 믿을 수 없다

- 한 번 이긴 것은 이긴 것이 아니다, 반복 실험과 변수 통제는 이후 **실험 설계**에서 본격적으로 다룬다

6. seed를 바꿔 다섯 번 돌려 보면



seed 0~4로 다섯 번씩 돌린 실측, 세로축 96.9~97.6%로 확대

- SGD는 **97.06~97.25%**, Adam은 **97.37~97.50%**로, 다섯 번 모두 Adam이 높았다
- 평균은 97.15% 대 97.43%, **+0.28%p**다, 반복해야 '이 조건에서는 Adam이 낫다'고 말할 수 있다
- seed가 바뀌면 초기 weight와 배치 순서가 바뀌어, 같은 설정이어도 결과가 조금씩 다르다

6. 정리, 과제, 다음 주

오늘의 구성 요소	역할	핵심 한 줄
nn.CrossEntropyLoss	분류 loss	정답 확률의 -log, 크게 틀릴수록 loss가 크게 커진다
loss.backward()	역전파	연쇄 법칙으로 79,510개의 grad를 한 번에 계산
opt.zero_grad(), opt.step()	갱신	grad 지우기, SGD는 w ← w − lr × grad
train(), eval(), no_grad	학습과 검증 루프	epoch마다 기록하고 학습 곡선과 간격으로 진단
Baseline	비교 기준	조건 고정, 검증으로 선택, 테스트로 보고

오늘 연 학습 과정, CNN도 같은 학습 함수와 검증 함수를 그대로 쓴다

- **다음 시간에는 같은 학습 함수와 검증 함수에 모델만 CNN으로 바꿔 오늘의 Baseline을 넘는다**